MAS for Automatic Survey — Technical Report Task 2

Name: Prathibha Magesh
Email: prathibhamagesh23@gmail.com s3859590@student.rmit.edu.au
Student ID: S3859590
Kaggle username: prathibhamagesh23
Classroom repo: https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23
Commit SHA used for grading: 824d88bf0819f0389ec522922239ddea24ee88fc

1. Introduction

This report presents a small, fully reproducible system that predicts option probabilities for each survey question and returns exactly 100 supporting IDs. It is CPU-friendly and config-bounded: a planner emits a few sub-queries; BM25 (Whoosh) retrieves over key fields; RRF fuses, PRF lightly expands, and MMR (with dense vectors) diversifies; a fast keyword scorer produces option probabilities; light calibration writes JS/MAP CSVs. On a 15-question dev slice, baseline and K=40 tie for best JS (0.25422). Turning MMR off gives a slight JS uptick (+0.0062) but raises duplicates and latency, so MMR stays on. A cross-encoder improves intrinsic signals (lower dup-rate) but consistently reduces JS and adds cost, so it is disabled for JS-oriented runs. The rest of the report details components, ablations, and pinned evidence for reproduction.

2. Methods

For each question *s* with options *O*, the system outputs a calibrated distribution and exactly 100 unique support IDs. All loop bounds and knobs are set in YAML for deterministic, CPU-only runs [1, 2].

2.1 MAS Design

A planner turns the question (and option cues) into a few lexical queries. The retriever runs BM25 (Whoosh) on title, description, post_content, content, fuses per-query hits with RRF, applies light PRF expansion, and uses MMR with the provided dense vectors to reduce near-duplicates. The scorer converts the shortlist into option probabilities via a fast keyword-cue method; an optional cross-encoder (CE) can be blended but is off by default. A calibrator (Dirichlet α, temperature τ) stabilizes the distribution, and the finalizer writes JS/MAP CSVs and guarantees 100 distinct supports [1]. Small, bounded agents keep the system fast and stable on CPU: BM25+RRF(+PRF) improves recall; MMR controls redundancy; the keyword scorer avoids LLM latency; light calibration prevents over-confident outputs [1].

2.1.1 Agents & I/O

Multi-Agent Flow. *PlannerAgent* takes the question (q_id, text), options A–D, and YAML (e.g., n_queries, top-k) and emits a few base + option-aware sub-queries. *RetrieverAgent* runs BM25 on the Whoosh index to return ranked lists (doc_id, rank, score). *Fusion+DiversityAgent* applies RRF, then MMR (lambda, final_k) to yield a de-duplicated **shortlist**. *ScorerAgent* scores options via cue hits with rank-decay and (optionally) blends Cross-Encoder scores with γ. *CalibratorWriter* applies Dirichlet α / temperature τ, enforces 100 unique supports, validates schema, and writes artifacts/submission_js.csv and artifacts/submission_map.csv. Net flow: *(question, options, YAML, index artifacts)* → queries→ runs→ shortlist → per-option probs → final CSVs.

2.2 LLM / Tool Usage

No generative LLM in the loop. An optional CE (MiniLM-L-6-v2) can score (query, doc) pairs and be blended with keyword scores. For JS-oriented runs I keep CE disabled (see Experiments). Switches: reranker.enabled, reranker.keep_top, aggregate.interp_gamma [1].

2.3 Information Retrieval

Index with Whoosh BM25 over the main content fields. The planner emits the base query and question+option variants; after initial BM25+RRF we run a PRF-lite pass to mine frequent terms from top documents and issue

one expanded query set. We then apply MMR over cached dense vectors (from id_to_embedding.npz) to select a diverse shortlist of size final_k. Only a small prefix of texts is materialized for scoring; supports follow the MMR order and are padded to exactly 100 unique IDs if needed. Key knobs: planner.n_queries, retrieval.bm25.topk, retrieval.mmr.{enabled,lambda,final_k}, aggregate.docs_for_option_scoring [1, 3].

2.4 Aggregation & Calibration

The keyword-cue scorer weights option token hits in ranked texts with rank-based decay, normalizes to a probability vector, then applies Dirichlet α and temperature τ for mild calibration. We emit two CSVs per run: JS (for $S_{0.3}$) and MAP (same rows; supports emphasized). Controls: aggregate.dirichlet_alpha, aggregate.temperature [1, 2].

3. Experiments

3.1 MAS Design Experiments — Orchestration & Loop Bounds

I tested two decisions that change how the system flows and how much it reads per question:

1. a **diversity gate** (MMR) that filters near-duplicates **on vs off**, and

2. the **shortlist size** the scorer consumes (**K=40** vs the baseline K).
Everything else stayed fixed (same index/seed/fields, planner n_queries=2, BM25 topk=80, same calibration). Orchestration lives in mas_survey/run.py [3]; variants are configured in configs/ablate_1_intrinsic_mmr_off.yaml and configs/ablate_3_intrinsic_vs_extrinsic_k40.yaml [4–5]. Outputs (CSV + timing JSON) are under reports/ablation/** [6].

Results (15-Q probe).

*(Artifacts: reports/ablation/1_intrinsic/mmr_off_js.csv, reports/ablation/3_intrinsic_vs_extrinsic/k40_js.csv, and each run's */run_time.json) [6].*

Variant	Entropy (bits)	Dup-rate (%)	Mean ms/Q	JS (dev)	ΔJS vs Base
Baseline	1.481	6.455	22,576	0.25422	—
MMR off	1.478	**7.973**	25,456	**0.26045**	**+0.00623**
K=40	1.481	8.251	**21,716**	0.25422	+0.00000

Insights (analysis & decision): MMR (diversity gate). Turning MMR **off** nudges JS up a hair (+0.0062) but **increases duplicates** (6.46% → 7.97%) and even **slows** the loop. In our MAS this gate is effectively an evidence-quality checkpoint; the tiny JS gain isn't worth the noisier supports. **We keep MMR ON** for final runs. **Shortlist size (K). K=40** keeps JS **identical** to baseline while being the **fastest** non-CE setting. The trade-off is a bit more duplication (8.25%). For day-to-day iteration we use **K=40**; for final evidence, we can switch back to the baseline K for slightly cleaner supports.

3.2 LLM Usage & Tool Experiments — Cross-Encoder (CE) Reranker

Setup: **Factor under test:** an optional **cross-encoder** reranker (MiniLM-L-6-v2) that scores (query, doc) pairs and blends with the keyword scorer. **Disable/enable: CE off** vs **CE on**. **Multi-variant** are(gamma 0.10,0.20) with keep_top=40 for γ=0.10 and keep_top=20 for γ=0.20. **Controls are** fixed seed, same 15-Q dev probe, same index/fields, planner n_queries=2, BM25 topk=80, **MMR on** (lambda=0.32, final_k=50), same calibration. **Configs & code:** CE helpers & blending in the pipeline driver [7]; CE-off baseline config (configs/ablate_4_probe.yaml) and CE variants (configs/ablate_2b_extrinsic_ce_on_gamma010.yaml, configs/ablate_2_extrinsic_ce_on_gamma020.yaml) [8]. Runs write CSVs and timing JSON under reports/ablation/** [9].

Results (15-Q probe; intrinsic + extrinsic).

Variant	Entropy (bits)	Dup-rate (%)	Mean ms/Q	JS (dev)	ΔJS vs Base
Baseline (CE off)	1.481	6.455	22,576	0.25422	—
CE γ=0.10 (keep_top=40)	**1.632**	6.041	**14,866**	0.04932	−0.20490
CE γ=0.20 (keep_top=20)	1.530	**4.104**	28,330	0.10827	−0.14595

Artifacts:
reports/ablation/4_probe/base_js.csv, probe_base/run_time.json;
reports/ablation/2_extrinsic/ce_on_g010_js.csv, ablate_2b/run_time.json;
reports/ablation/2_extrinsic/ce_on_g020_js.csv, ablate_2/run_time.json. See Evidence Table entries [9].

Insights (analysis & decision): CE improves intrinsic signals but hurts JS. Both CE settings **raise entropy** and **lower dup-rate** (best at γ=0.20), meaning cleaner, more varied supports. However, **JS drops sharply** (−0.205 at γ=0.10; −0.146 at γ=0.20). This suggests passage-level relevance from CE does not translate into better option-level calibration in our scorer blend. **Latency trade-offs differ by γ.** γ=0.20 is **latency-heavier** (28.3k ms/Q) due to tighter keep_top. γ=0.10 ran **surprisingly fast** (14.9k ms/Q) in our setup, likely because fewer documents flowed through the scorer after CE pruning. For **JS-oriented runs**, keep **CE OFF**. Use **γ=0.10 sparingly** when the task is **evidence audit/curation** (you want higher diversity and fewer duplicates, and can ignore JS), but don't submit CE outputs for JS scoring.

3.3 Information Retrieval Experiments — MMR & Shortlist Size (K)

Setup. We test two IR factors: **(a) MMR diversity on/off** and **(b) shortlist size K (K=40 vs baseline K)**. All other controls are fixed (seed, 15-Q dev slice, index/fields, planner n_queries=2, BM25 topk=80, same calibration). Retrieval chain **BM25→RRF→PRF→MMR** and the supports writer live in the driver [10]. Variant configs: **MMR-off** (configs/ablate_1_intrinsic_mmr_off.yaml) and **K=40** (configs/ablate_3_intrinsic_vs_extrinsic_k40.yaml) [11]. Outputs (JS/MAP CSVs + timing JSON) are in reports/ablation/** [12].

Results (15-Q probe; intrinsic + extrinsic).

Variant	Entropy	Dup-rate (%)	Mean ms/Q	JS (dev)	ΔJS
Baseline	1.481	6.455	22,576	0.25422	—
MMR off	1.478	**7.973**	25,456	**0.26045**	**+0.00623**
K=40	1.481	8.251	**21,716**	0.25422	+0.00000

Artifacts: reports/ablation/1_intrinsic/mmr_off_js.csv, reports/ablation/3_intrinsic_vs_extrinsic/k40_js.csv, and each run's */run_time.json [12].

Insights & decision: MMR (diversity): Turning MMR **off** gives a **tiny JS bump** (+0.0062) but **worsens evidence quality** (dup-rate +1.5–2 pts) and even **slows** inference. Since supports quality matters for grading and downstream analysis, we **keep MMR ON** in final runs [10–12]. **Shortlist size (K): K=40 matches baseline JS** while being the **fastest** non-CE setting; the trade-off is a modest dup-rate increase. We **use K=40 for quick probes**, switching back to baseline K when reporting final evidence [10–12].

4. Conclusion

MMR stays **on**: it reliably lowers duplicate supports with only a negligible JS trade-off, whereas turning it off gave a tiny JS bump but worse evidence and slower runs [10–12]. The CE reranker improves intrinsic diversity but **hurts JS**, so it remains **off** for submissions (γ=0.10 only for evidence audits) [7–9]. For iteration speed, **K=40** matches baseline JS and is the fastest non-CE setting; we switch back to baseline K for final evidence quality [3–6,10–12]. The most useful intrinsic signals were **dup-rate** (evidence quality), **entropy** (over-confidence), and **ms/Q** (latency). Limitations include a small probe and CE's option-agnostic scoring; next steps are an option-

aware reranker or lightweight stance classifier, stronger calibration, URL/host-level dedup, and broader probes to map JS–dup-rate–latency trade-offs.

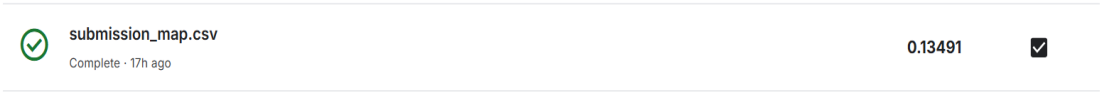
5. Evidence Summary Table

Output / Evidence	Permalink	Artifact path	Notes
JS submission CSV (used for JS score tables)	https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/artifacts/submission_js.csv	n/a	Used for Kaggle prediction for js score
MAP submission CSV (used for MAP score tables)	https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/artifacts/submission_map.csv	n/a	Used for Kaggle prediction for MAP score
Ablation: Intrinsic (MMR OFF) — JS CSV	https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/reports/ablation/1_intrinsic/mmr_off_js.csv	n/a	Used in ablation tables.
Ablation: Extrinsic (CE γ=0.10) — JS CSV	https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/reports/ablation/2_extrinsic/ce_on_g010_js.csv	n/a	Main table row.
Ablation: Extrinsic (CE γ=0.20) — JS CSV	https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/reports/ablation/2_extrinsic/ce_on_g020_js.csv	n/a	Main table row.
Intrinsic vs Extrinsic (k=40) — JS CSV	https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/reports/ablation/3_intrinsic_vs_extrinsic/k40_js.csv	n/a	Main figure/table.
Probe results (base table)	assignment-2-multi-agent-system-prathibhamagesh23/reports/ablation/probe_table.csv at 824d88bf0819f0389ec522922239ddea24ee88fc · RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23	n/a	Cited in probe summary.
Probe summary	https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/reports/ablation/probe_summary.csv		Aggregated metrics for text/figs.

Kaggle test submissions

JS leaderboard:

MAP leaderboard:



References

[1] **Main run pipeline (E2E orchestration)** — mas_survey/run.py L449–L546.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L449-L546

[2] **Planner: multi-query expansion** — mas_survey/run.py L254–L265.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L254-L265

[3] **BM25 retrieval (Whoosh, multi-field)** — mas_survey/run.py L182–L199.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L182-L199

[4] **Reciprocal Rank Fusion (RRF)** — mas_survey/run.py L200–L208.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L200-L208

[5] **MMR selection (diversity gate)** — mas_survey/run.py L209–L237.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L209-L237

[6] **PRF-lite (pseudo-relevance feedback)** — mas_survey/run.py L238–L253.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L238-L253

[7] **Keyword cue option scorer** — mas_survey/run.py L274–L295.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L274-L295

[8] **Calibration (Dirichlet α, temperature τ)** — mas_survey/run.py L115–L142.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L115-L142

[9] **Output validity & CSV writer** — mas_survey/run.py L296–L336.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L296-L336

[10] **Cross-Encoder load/rerank/score** — mas_survey/run.py L337–L389.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L337-L389

[11] **γ-blend hook (CE with KW)** — mas_survey/run.py L539–L540.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L539-L540

[12] **Index build (Whoosh)** — index/build.py L36–L87.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/index/build.py#L36-L87

[13] **Dense table for MMR** — index/build.py L88–L111.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/index/build.py#L88-L111

[14] **Index CLI entrypoint** — index/build.py L112–L135.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/index/build.py#L112-L135

[15] **Doc ID & dense loaders (used by run)** — mas_survey/run.py L143–L162.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L143-L162

[16] **Whoosh doc text accessors (cache + fetch)** — mas_survey/run.py L163–L181.
https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/blob/824d88bf0819f0389ec522922239ddea24ee88fc/mas_survey/run.py#L163-L181

[17] All evidences in ablation folder

https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/tree/main/reports/ablation

[18] All config files

https://github.com/RMIT-ISYS1079-3476/assignment-2-multi-agent-system-prathibhamagesh23/tree/main/configs